



# AI ENGINEERING

## PROJECT SCOPING DOC

### 0. Project Summary

- *Problem:*
- *Why it matters / impact:*
- *Proposed approach:*
- *Primary success metric:*
- *Key risks / unknowns:*

### 1. Problem Framing & Success Metrics

#### **Business / User Problem**

*Describe the underlying problem in plain language. Who is the user? What pain point or workflow does this solve? Why is AI engineering the right approach (vs. traditional software, rule-based systems, or classical ML)?*

#### **Goal**

*Specific, measurable goal(s). What does "solved" look like? Tie to user outcomes and timelines.*

#### **Stakeholders**

*List key people/teams and their roles/ownership (PM, Eng, Design, Data, Legal, etc.).*

#### **Prior Work**

*Summarize past attempts, related products/tools, or heuristic solutions. Include links if relevant.*

#### **Input and Output**

*A brief description of what data goes into the system, and what the system outputs. Also consider what data you will need to store, and where.*

#### **Constraints**

| Constraint            | Target | Notes                                                           |
|-----------------------|--------|-----------------------------------------------------------------|
| Latency (p50 / p99)   |        | <i>e.g., &lt; 2s for chat, &lt; 5s for research tasks</i>       |
| Cost per request      |        | <i>e.g., &lt; \$0.03 avg</i>                                    |
| Quality bar           |        | <i>e.g., 90%+ user satisfaction, &lt; 5% hallucination rate</i> |
| Privacy / compliance  |        | <i>e.g., no PII in logs, GDPR compliant</i>                     |
| Availability / uptime |        | <i>e.g., 99.5% uptime</i>                                       |

### Success Metrics

*Define metrics at three levels:*

- *User-facing metrics:* Task completion rate, user satisfaction (thumbs up/down, CSAT), time saved vs. manual process.
- *Technical metrics:* Response quality (LLM-as-judge scores, human ratings, accuracy on test cases), hallucination rate, format compliance rate.
- *System metrics:* Latency (p50/p99), cost per request, error rate, uptime.

### Checklist:

- ☐ I have strong evidence that the problem I'm trying to solve is an important problem to solve.
- ☐ I have strong evidence that an LLM-based approach is the right tool for this problem (vs. traditional software, rule-based, or classical ML).
- ☐ The goals and metrics align with user needs and/or team priorities.
- ☐ I have shared the scope with relevant stakeholders and flagged dependencies.
- ☐ I have documented key decisions and trade-offs in my README or decision log.

## 2. Prompt Engineering & Systematic Tracking

### Prompt Architecture

*Describe the overall prompt strategy. How many distinct prompts does your system use? What roles do they play (system prompt, few-shot examples, templates, output parsers)?*

### Prompt Organization

Describe how prompts are stored and versioned. Prompts should be treated as separate components — not hardcoded in application files.

### Example structure:

```
None
prompts/
├── system_prompts/
│   ├── assistant_v1.txt
│   ├── assistant_v2.txt
│   └── summarizer_v1.txt
├── few_shot_examples/
│   ├── classification_examples.json
│   └── query_reformulation_examples.json
└── templates/
    └── rag_response_template.txt
```

### Evaluation Framework

Describe your test set and evaluation approach for prompts.

| Evaluation Aspect    | Approach                                                                     |
|----------------------|------------------------------------------------------------------------------|
| Test set size        | <i>e.g., 100–300 test cases</i>                                              |
| Test set composition | <i>Cover different question types, difficulty levels, edge cases</i>         |
| Metrics              | <i>e.g., LLM-as-judge scores, BLEU/ROUGE, exact match, format compliance</i> |
| Evaluation tooling   | <i>e.g., manual review, PromptLayer, Langfuse, Weights &amp; Biases</i>      |
| Update cadence       | <i>How often is the test set refreshed with new failure patterns?</i>        |

### Checklist:

- ☐ Prompts are stored in separate versioned files, not hardcoded.
- ☐ I have a structured test set with diverse cases and expected outputs/criteria.
- ☐ I have a repeatable evaluation process for comparing prompt versions.
- ☐ Prompt changes are tested against the evaluation set before deployment.
- ☐ I have documented my prompt iteration history and key learnings.

### 3. Model Selection & Evaluation

#### Candidate Models

| Model | Provider | Cost<br>(input/output) | Latency<br>(est.) | Quality<br>Notes | License                                                           |
|-------|----------|------------------------|-------------------|------------------|-------------------------------------------------------------------|
|       |          |                        |                   |                  | <i>Ensure the<br/>license<br/>works for<br/>your use<br/>case</i> |
|       |          |                        |                   |                  |                                                                   |

#### Evaluation Approach

*Describe how you'll compare models. Run each candidate on the same test set and compare performance, cost, and latency using a consistent rubric.*

#### Model Routing (if applicable)

*If using multiple models, describe the routing strategy. e.g., a cheap/fast model classifies query difficulty and routes simple queries to a small model and complex queries to a larger one.*

#### Checklist:

- ☐ I have tested at least 2–3 models from different providers/sizes on my test set.
- ☐ I have compared performance, cost, and latency in a structured way.
- ☐ I have ensured the model licenses are appropriate for my use case.
- ☐ I have documented the rationale for my final model choice(s).
- ☐ If using model routing, I have tested and documented the routing logic.

### 4. RAG Implementation

*Skip this section if your project does not use RAG.*

#### Data Sources

| Source | Type | Volume | Update<br>Frequency | Notes |
|--------|------|--------|---------------------|-------|
|        |      |        |                     |       |

#### Chunking Strategy

| Strategy Tested | Chunk Size | Overlap | Retrieval Accuracy | Notes |
|-----------------|------------|---------|--------------------|-------|
|                 |            |         |                    |       |

*Test multiple strategies (fixed-size, semantic, recursive, etc.) on your specific data and measure retrieval accuracy.*

### Embedding Model & Vector Store

| Component       | Choice                                                            | Rationale |
|-----------------|-------------------------------------------------------------------|-----------|
| Embedding model | <i>e.g., OpenAI text-embedding-3-small, Sentence Transformers</i> |           |
| Vector store    | <i>e.g., ChromaDB, FAISS, Pinecone, Weaviate</i>                  |           |

### Retrieval Strategy

*Describe the search approach: semantic similarity, hybrid search (keyword + semantic), re-ranking, query expansion, etc. Document what you tested and the improvement each technique provided.*

### RAG Evaluation

| Metric                              | Description                                                | Baseline | Current |
|-------------------------------------|------------------------------------------------------------|----------|---------|
| Retrieval accuracy (Precision@K)    | % of retrieved chunks that are relevant                    |          |         |
| Retrieval recall (Recall@K)         | % of relevant chunks retrieved                             |          |         |
| Answer accuracy (given good chunks) | Does the LLM produce correct answers with correct context? |          |         |
| End-to-end accuracy                 | Does the whole system produce correct answers?             |          |         |

**Checklist:**

- ☐ I have tested multiple chunking strategies and documented results.
- ☐ I have evaluated at least two embedding models.
- ☐ I have a retrieval test set (20–30+ questions with known answer locations).
- ☐ I can isolate whether failures come from retrieval or generation.
- ☐ I have documented the improvement from each retrieval enhancement (re-ranking, hybrid search, query expansion, etc.).

## 5. Agent Systems

*Skip this section if your project does not use agents.*

**Agent Architecture**

*Describe the agent's purpose, the framework used (LangGraph, CrewAI, custom function calling, etc.), and the high-level control flow.*

**Tools**

| Tool | Purpose | Input / Output | Error Handling |
|------|---------|----------------|----------------|
|      |         |                |                |

**Safety & Robustness**

| Concern                  | Approach                                                                               |  |
|--------------------------|----------------------------------------------------------------------------------------|--|
| Error handling           | <i>What happens when a tool call fails? Retry? Fallback? Graceful error message?</i>   |  |
| Security                 | <i>Input validation, sandboxed execution, access control</i>                           |  |
| Infinite loop prevention | <i>Max iterations, loop detection, cost budgets</i>                                    |  |
| Guardrails               | <i>Content filtering, output validation, human-in-the-loop for high-stakes actions</i> |  |

| Concern         | Approach |                                               |
|-----------------|----------|-----------------------------------------------|
| Cost monitoring |          | <i>Ensure the agents don't bankrupt you 😊</i> |

### Agent Evaluation

| Test Type                              | # Tests | Description                                                                           |
|----------------------------------------|---------|---------------------------------------------------------------------------------------|
| Unit tests (individual tools)          |         |                                                                                       |
| Integration tests (complete workflows) |         |                                                                                       |
| Adversarial tests                      |         |                                                                                       |
| Representative task suite              |         | <i>Simple to complex tasks; measure task completion rate, avg steps to completion</i> |

### Checklist:

- ☐ Each tool has unit tests and documented input/output contracts.
- ☐ I have explicit error handling for all tool failures.
- ☐ I have safeguards against infinite loops and runaway costs.
- ☐ I have a representative task test suite with measured completion rates.
- ☐ I have logged and analyzed agent traces to identify failure patterns.

## 6. Deployment & User Interface

### API Design

| Aspect         | Approach                                                       |
|----------------|----------------------------------------------------------------|
| Framework      | <i>e.g., FastAPI, Flask</i>                                    |
| Streaming      | <i>Yes/No — important for perceived latency with LLM calls</i> |
| Authentication | <i>e.g., API key, OAuth</i>                                    |
| Rate limiting  | <i>Approach to prevent abuse</i>                               |

| Aspect         | Approach                                                         |
|----------------|------------------------------------------------------------------|
| Error handling | <i>How API failures (LLM timeouts, rate limits) are surfaced</i> |

### User Interface

| Aspect             | Approach                                                   |
|--------------------|------------------------------------------------------------|
| UI framework       | <i>e.g., Streamlit, Gradio, React/Next.js</i>              |
| Key interactions   | <i>What can the user do? What does the flow look like?</i> |
| Feedback mechanism | <i>e.g., thumbs up/down, report bad responses</i>          |

### Infrastructure

| Aspect           | Approach                                                   |
|------------------|------------------------------------------------------------|
| Hosting          | <i>e.g., AWS, GCP, Azure</i>                               |
| Containerization | <i>e.g., Docker</i>                                        |
| CI/CD            | <i>e.g., GitHub Actions — auto-test and deploy on push</i> |

### Checklist:

- ☐ My API handles streaming, errors, and rate limiting.
- ☐ My UI is accessible and provides a way for users to give feedback.
- ☐ My application is containerized with Docker.
- ☐ I have CI/CD set up so tests run automatically on push.
- ☐ I have a live demo link or clear setup instructions in my README.

## 7. System Monitoring & Error Analysis

### Component-Level Monitoring

| Component      | Metrics to Track                                                                              |
|----------------|-----------------------------------------------------------------------------------------------|
| Prompts        | Response quality scores, format compliance, refusal rate, avg response length                 |
| RAG            | Retrieval confidence scores, # chunks retrieved, source diversity, retrieval latency          |
| Agents         | Task completion rate, avg steps to completion, tool success rates, error types, cost per task |
| Overall system | End-to-end success rate, user satisfaction, latency, cost per request, error rate, uptime     |

## Logging

*At minimum, log for each request: timestamp, user query, components used (which chunks retrieved, which model, which prompt version), response, latency, cost, and any errors.*

| Logging Approach | Tool/Method                                                                        |
|------------------|------------------------------------------------------------------------------------|
| Basic            | <i>File or SQLite database</i>                                                     |
| Advanced         | <i>e.g., Langfuse, LangSmith, Weights &amp; Biases, custom observability stack</i> |

## Error Analysis Process

*Describe how you regularly review failures, categorize error types, and feed learnings back into improving prompts, RAG configuration, or agent logic.*

### Checklist:

- ☐ I have structured logging for every request.
- ☐ I track cost, latency, and quality metrics over time.
- ☐ I have a process for reviewing and categorizing errors.
- ☐ I have a feedback loop: errors inform prompt/RAG/agent improvements.
- ☐ I have alerting for critical metric regressions (if applicable).

## 8. Fine-Tuning (Optional / Advanced)

*Only pursue fine-tuning if you've optimized everything else and have a clear reason.*

### Use Case for Fine-Tuning

*Why fine-tuning is appropriate here (e.g., consistent output formatting, matching a larger model's performance with a smaller model, domain-specific language, embedding model tuning for RAG).*

### Approach

| Aspect                 | Details                                                              |
|------------------------|----------------------------------------------------------------------|
| Training data          | <i># examples, how they were created, quality assurance</i>          |
| Base model             | <i>Which model are you fine-tuning? Why this model?</i>              |
| Baseline               | <i>Performance of best prompt-only approach on held-out test set</i> |
| Versions trained       | <i>Different data sizes, epochs, learning rates</i>                  |
| Performance comparison | <i>Does fine-tuned model beat baseline? By how much?</i>             |

### Checklist:

- ☐ I have exhausted prompt engineering and RAG optimizations before resorting to fine-tuning.
- ☐ I have a high-quality training dataset (100–500+ examples).
- ☐ I have a baseline to compare against.
- ☐ I have trained multiple versions and tracked results.
- ☐ I have documented whether fine-tuning was worth the effort.

## 9. Code Quality & Repository Structure

### Project Structure

```
None
your-project/
├─ src/
```

```

|   ├── prompts/           # Prompt templates and versions
|   ├── rag/               # RAG components (chunking, retrieval, indexing)
|   ├── agents/           # Agent logic and tools
|   ├── models/           # Model selection and evaluation
|   └── monitoring/        # Logging and metrics
├── api/                   # FastAPI application
├── frontend/              # UI code
├── tests/                 # Unit and integration tests
├── evaluation/            # Test sets and evaluation scripts
├── data/                  # Sample data or data configs
├── deployment/            # Docker, cloud config files
├── docs/                  # Architecture diagrams, decision log
├── .env.example           # Example environment variables
├── requirements.txt        # Dependencies
├── Dockerfile
└── README.md

```

## Code Standards

- ☐ Type hints on all functions.
- ☐ Docstrings on all functions (what it does, parameters, returns).
- ☐ No hardcoded values — API keys, model names, chunk sizes in config/env vars.
- ☐ Modular code in `.py` files, not monolithic notebooks.

## README Contents

- ☐ Clear overview of problem and solution.
- ☐ Architecture diagram showing how components connect.
- ☐ Key decisions and trade-offs (what you tested, what you learned).
- ☐ Setup instructions (clone → run).
- ☐ Performance metrics and evaluation results.
- ☐ Live demo link (if deployed).

## 10. Project Timeline & Milestones

| Milestone                         | Target Date | Status |
|-----------------------------------|-------------|--------|
| Problem scoping & design complete |             |        |
| Prompt engineering baseline       |             |        |

| Milestone                               | Target Date | Status |
|-----------------------------------------|-------------|--------|
| RAG pipeline v1                         |             |        |
| Agent system v1 (if applicable)         |             |        |
| Evaluation framework complete           |             |        |
| Deployment & UI                         |             |        |
| Monitoring & observability              |             |        |
| Fine-tuning experiments (if applicable) |             |        |
| Final polish, README, demo              |             |        |

## 11. Appendix

- *Links to repos, notebooks, dashboards, evaluation results, etc.*
- *Architecture diagram(s)*
- *Glossary of terms/metrics*
- *Decision log — key choices made and why*